

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Объектно-ориентированное программирование
Отчет по курсовой работе «Телефонный справочник»

Студент,
группы 5130201/30003

_____ Мартыненко А. П.

Преподаватель

_____ Кирпиченко С. Р.

«_____» _____ 2024г.

Санкт-Петербург, 2024

Содержание

Введение	3
1 Постановка задачи	4
2 Реализация	6
2.1 Класс MainWindow	6
2.2 Класс InputWidget	11
2.3 Класс Logger	16
3 Тестирование приложения	18
Заключение	23
Список использованной литературы	24
Приложение А	25

Введение

Суть приложения заключается в добавлении новых контактов в телефонную книгу. С помощью данного телефонного справочника можно быстро узнать ФИО, адрес, дату рождения, email и номера телефонов человека. Дополнительно приложение позволяет создавать новые записи, удалять имеющиеся, сортировать и производить поиск по записям. Для хранения информации используется таблица со столбцами в соответствии с записываемой информацией. Приложение реализовано на языке C++ с использованием фреймворка Qt.

Qt для C++ - это кроссплатформенный фреймворк для разработки программного обеспечения. Он является полностью объектно-ориентированным и состоит из множества модулей, выполняющих определенные задачи. С помощью Qt можно создавать графические интерфейсы и приложения.

Разработка графических приложений отличается от консольных. Графические приложения - это программы, которые предоставляют графический интерфейс с визуальными элементами управления (кнопками, текстовыми полями, изображениями). Такие приложения интуитивно понятны пользователю и просты в использовании.

1 Постановка задачи

Исходные данные: в телефонном справочнике должны быть корректные поля: имя, фамилия, отчество, адрес, дата рождения, email, телефонные номера (рабочий, домашний, служебный) в любом количестве. При запуске приложения выгружаются ранее записанные данные из файла.

Цель: реализация телефонного справочника.

Задачи:

- 1) Создание интерфейса для хранения и взаимодействия с обязательными полями: ФИО, адрес, дата рождения, email, телефонные номера (служебный, домашний, рабочий).
- 2) Реализация проверки всех вводимых данных на корректность при помощи регулярных выражений по следующим параметрам:
 - Фамилия, Имя или Отчество должны содержать только буквы и цифры различных алфавитов, а также дефис и пробел, но при этом должны начинаться только на заглавные буквы, и не могли бы оканчиваться или начинаться на дефис. Все незначимые пробелы перед и после данных должны удаляться.
 - Телефон должен быть записан в международном формате, а храниться внутри как последовательность цифр. Один из вариантов записи телефона: префикс _выхода_ на международную _линию код_ страны код_города телефон. Пример, +7 812 1234567, 8(800)123-1212, + 38 (032) 123 11 11.
 - Дата рождения должна быть меньше текущей даты, число месяцев в дате должно быть от 1 до 12, число дней от 1 до 31, причем должно учитываться различное число дней в месяце и високосные года.
 - E-mail должен содержать в себе имя пользователя состоящее из латинских букв и цифр, символ разделения пользователя и имени домена(@), а также сам домен состоящий из латинских букв и цифр. Все незначащие пробелы (включая пробелы перед и после символа @) должны быть удалены.
- 3) Реализация чтения из файла и записи в файл формата txt данных.
- 4) Реализация возможности добавления новых записей в телефонный справочник. При добавлении новой записи должна появляться новая, изначально пустая, строка, в которой пользователь должен заполнить все обязательные поля.
- 5) Реализация удаления имеющейся записи таблицы, выделенной пользователем.

6) Реализация редактирования любого выделенного поля. При редактировании должна происходить валидация данных, чтобы сохранялись только корректные данные.

7) Реализация сортировки отображаемых данных по любому из существующих полей.

8) Реализация поиска записей в таблице с помощью ввода поискового значения в однострочный текстовый редактор. Поиск должен осуществляться по частичному совпадению среди всех полей. Ячейки, не удовлетворяющие запросу, необходимо скрыть.

9) Реализация двухуровневого логирования данных с записью в текстовый файл.

2 Реализация

2.1 Класс MainWindow

Класс MainWindow публично наследуется от QMainWindow. QMainWindow является подклассом QWidget и предоставляет более сложную структуру для создания главных окон приложений. С его помощью можно создать композицию виджетов и удобно организовать разные части интерфейса в единую систему, наладить связь между ними. Это позволяет классу MainWindow выступать в роли контроллера, передающего необходимые параметры из класса InputWidget.

Конструктор MainWindow создает таблицу QTableWidgetItem для хранения информации о контактах, кнопки управления: добавление (Add note), редактирование (Edit note) и удаление (Delete note), строку поиска; размещает все эти элементы в вертикальном макете QVBoxLayout. В конструкторе настраиваются соединения для обработки различных событий, задаются цвета кнопок и заголовков таблиц. Также он загружает данные из файла с контактами.

Реализация конструктора приведена в листинге 1.

Листинг 1. Реализация конструктора MainWindow

```
1 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent){
2     setFixedSize(1000, 600);
3     tableWidget = new QTableWidgetItem;
4     tableWidget->setColumnCount(ColumnCount);
5     tableWidget->setHorizontalHeaderLabels({"Surname", "Name", "Second
6     name", "Date", "Address", "Phone", "Email"});
7     tableWidget->horizontalHeader()->setSectionResizeMode(QHeaderView::
8     Stretch);
9
10    tableWidget->verticalHeader()->setStyleSheet("QHeaderView::section {
11    background-color: #B3DCFD; color: black; }");
12    tableWidget->horizontalHeader()->setStyleSheet("QHeaderView::section {
13    background-color: #B3DCFD; color: black; }");
14    QPalette palette = tableWidget->palette();
15    palette.setColor(QPalette::Highlight, QColor("#33FF99"));
16    palette.setColor(QPalette::HighlightedText, QColor("black"));
17    tableWidget->setPalette(palette);
18
19    QPushButton* addButton = new QPushButton("Add note");
20    addButton->setStyleSheet(R"(
21    QPushButton {
22        background-color: #3399ff;
23        color: white;
24    }
25    QPushButton:hover {
26        background-color: #02F7FD;
27        color: black;
28    }
29    QPushButton:pressed {
30        background-color: #004080;
31    }
32    )");
```

```

29     QPushButton* editButton = new QPushButton("Edit note");
30     editButton->setStyleSheet(R"(
31     QPushButton {
32         background-color: #3399ff;
33         color: white;
34     }
35     QPushButton:hover {
36         background-color: #24DB4E;
37         color: black;
38     }
39     QPushButton:pressed {
40         background-color: #004080;
41     }
42     )");
43     QPushButton* deleteButton = new QPushButton("Delete note");
44     deleteButton->setStyleSheet(R"(
45     QPushButton {
46         background-color: #3399ff;
47         color: white;
48     }
49     QPushButton:hover {
50         background-color: #EF9B10;
51         color: black;
52     }
53     QPushButton:pressed {
54         background-color: #DF9820;
55     }
56     )");
57     QLabel* searchLabel = new QLabel("Search Line:");
58     searchEdit = new QLineEdit;
59
60     QWidget* widget = new QWidget;
61     QVBoxLayout* layout = new QVBoxLayout(widget);
62     layout->addWidget(tableWidget);
63     layout->addWidget(addButton);
64     layout->addWidget(editButton);
65     layout->addWidget(deleteButton);
66     layout->addWidget(searchLabel);
67     layout->addWidget(searchEdit);
68
69     setCentralWidget(widget);
70
71     connect(tableWidget->horizontalHeader(), &QHeaderView::sectionClicked,
72         this, &MainWindow::sortEntries);
73     connect(addButton, &QPushButton::clicked, this, &MainWindow::addEntry);
74     connect(editButton, &QPushButton::clicked, this, &MainWindow::
75         editEntry);
76     connect(deleteButton, &QPushButton::clicked, this, &MainWindow::
77         deleteEntry);
78     connect(searchEdit, &QLineEdit::textChanged, this, &MainWindow::
79         searchEntry);
80
81     loadFromFile();
82 }

```

Деструктор MainWindow сохраняет данные таблицы в файл с помощью функции saveToFile при завершении приложения, гарантируя, что информация не будет утеряна. Реализация деструктора приведена в листинге 2.

Листинг 2. Реализация деструктора MainWindow

```

1   MainWindow::~MainWindow() {
2       saveToFile();
3   }

```

Метод `addEntry` предназначен для добавления новой записи в таблицу. Он отображает диалоговое окно `InputWidget`, валидирует корректность и при успешном вводе добавляет новую строку с введенными данными. Также он запрещает редактирование ячеек через нажатие по таблице. При успешном добавлении сообщение об этом логируется с помощью функции класса `Logger`.

Реализация метода `addEntry` приведена в листинге 3.

Листинг 3. Реализация метода addEntry

```

1 void MainWindow::addEntry() {
2     InputWidget inputWidget(this);
3     if (inputWidget.exec() == QDialog::Accepted) {
4         tableWidget->insertRow(tableWidget->rowCount());
5         for (int i = Surname; i < ColumnCount; ++i) {
6             QTableWidgetItem* item = new QTableWidgetItem(inputWidget.getData(i)
7         );
8             item->setFlags(item->flags() & ~Qt::ItemIsEditable);
9             tableWidget->setItem(tableWidget->rowCount() - 1, i, item);
10        }
11        Logger::log("INFO", "A new note was added to the table.");
12    }
13 }

```

Метод `editEntry` применяется для редактирования существующей выделенной записи. Для этого он загружает данные из выделенной строки в диалоговое окно `InputWidget`, обновляя содержимое ячеек после проверки корректности изменений. При успешном изменении сообщение об этом логируется с помощью функции класса `Logger`.

Реализация метода `editEntry` приведена в листинге 4.

Листинг 4. Реализация метода editEntry

```

1 void MainWindow::editEntry() {
2     int row = tableWidget->currentRow();
3     if (row == -1) return;
4     InputWidget inputWidget(this);
5     for (int i = Surname; i < ColumnCount; ++i) {
6         inputWidget.setData(i, tableWidget->item(row, i)->text());
7     }
8     if (inputWidget.exec() == QDialog::Accepted) {
9         if (inputWidget.validateData()) {
10            for (int i = Surname; i < ColumnCount; ++i) {
11                tableWidget->item(row, i)->setText(inputWidget.getData(i));
12            }
13            Logger::log("INFO", "A note was successfully edited.");
14        }
15    }
16 }

```

Метод `deleteEntry` удаляет выделенную строку из таблицы. При ошибке или успешном удалении запись об этом логируется с разными уровнями с

помощью функции класса Logger. Реализация метода deleteEntry приведена в листинге 5.

Листинг 5. Реализация метода deleteEntry

```
1 void MainWindow::deleteEntry() {
2     int row = tableWidget->currentRow();
3     if (row != -1)
4     {
5         tableWidget->removeRow(row);
6         Logger::log("INFO", "A row was successfully deleted from the table
7     .");
8     }
9     else {
10        Logger::log("WARNING", "Attempt to delete a row when no row is
        selected or no row exists.");
    }
```

Метод searchEntry реализует поиск записей по таблице. Он получает текст запроса из строки поиска, проверяет таблицу построчно на совпадения и в случае несоответствия строки запросу скрывает ее.

Реализация метода searchEntry приведена в листинге 6.

Листинг 6. Реализация метода searchEntry

```
1 void MainWindow::searchEntry() {
2     QString query = searchEdit->text();
3     for (int i = 0; i < tableWidget->rowCount(); ++i) {
4         bool match = false;
5         for (int j = 0; j < tableWidget->columnCount(); ++j) {
6             if (tableWidget->item(i, j)->text().contains(query, Qt::
7             CaseInsensitive)) {
8                 match = true;
9                 break;
10            }
11        }
12        tableWidget->setRowHidden(i, !match);
13    }
```

Метод sortEntries сортирует записи в таблице по указанному (выделенному) столбцу по убыванию или возрастанию. После каждого вызова меняет порядок сортировки.

Реализация метода sortEntries приведена в листинге 7.

Листинг 7. Реализация метода sortEntries

```
1 void MainWindow::sortEntries(int column) {
2     tableWidget->sortItems(column, ascendingOrder ? Qt::AscendingOrder : Qt
3     ::DescendingOrder);
4     ascendingOrder = !ascendingOrder;
5 }
```

Метод loadFromFile загружает данные из файла в таблицу. С помощью кодировки UTF-8 он распознает как русский, так и английский текст. Этот метод разделяет данные строки по символу *, и, в случае соответствия строки ожидаемому формату, добавляет данные в таблицу, дополнительно запрещая редактирование путем нажатия на ячейку.

Реализация метода loadFromFile приведена в листинге 8.

Листинг 8. Реализация метода loadFromFile

```
1 void MainWindow::loadFromFile() {
2     QFile file("contacts.txt");
3     if (file.open(QIODevice::ReadOnly)) {
4         QTextStream in(&file);
5         in.setCodec("UTF-8");
6         while (!in.atEnd()) {
7             QStringList data = in.readLine().split("*");
8             if (data.size() == ColumnCount) {
9                 int row = tableWidget->rowCount();
10                tableWidget->insertRow(row);
11                for (int i = Surname; i < ColumnCount; ++i) {
12                    QTableWidgetItem* item = new QTableWidgetItem(data[i]);
13                    item->setFlags(item->flags() & ~Qt::ItemIsEditable);
14                    tableWidget->setItem(row, i, item);
15                }
16            }
17        }
18        file.close();
19    }
20 }
```

Метод saveToFile сохраняет данные из таблицы в файл. Так как в таблице содержатся уже провалидированные данные, он проверяет строки таблицы только на полноту и записывает данные каждой полной строки в файл, разделяя их символом *. Использование кодировки UTF-8 позволяет корректно обрабатывать данные различных алфавитов.

Реализация метода saveToFile приведена в листинге 9.

Листинг 9. Реализация метода saveToFile

```
1 void MainWindow::saveToFile() {
2     QFile file("contacts.txt");
3     if (file.open(QIODevice::WriteOnly)) {
4         QTextStream out(&file);
5         out.setCodec("UTF-8");
6         bool isRowComplete = true;
7         for (int i = 0; i < tableWidget->rowCount(); ++i) {
8             QStringList rowData;
9             for (int j = 0; j < tableWidget->columnCount(); ++j) {
10                QTableWidgetItem* item = tableWidget->item(i, j);
11                if (item && !item->text().isEmpty()) {
12                    rowData << item->text();
13                }
14                else {
15                    isRowComplete = false;
16                    break;
17                }
18            }
19            if (isRowComplete) {
20                out << rowData.join("*") << "\n";
21            }
22        }
23        file.close();
24    }
25 }
```

2.2 Класс InputWidget

Класс InputWidget публично наследуется от QWidget, который является базовым классом для всех виджетов в библиотеке Qt и обеспечивает основные функции для создания и управления графическим интерфейсом. Также InputWidget наследуется от QDialog, который предоставляет возможность создания модальных диалогов, т.е. пользователь не может взаимодействовать с другими окнами приложения, пока не завершит ввод. Такой тип окон хорошо подходит для временных окон (например, для ввода данных), потому что они не требуют постоянного взаимодействия.

Конструктор InputWidget создает QLineEdit поля для ввода ФИО, адреса, email; QDateEdit для ввода даты; кнопку для подтверждения корректности введенных данных okButton. Для добавления новых номеров телефона предназначена кнопка addPhone. Поля размещаются с использованием макета QFormLayout. В конструкторе настраиваются соединения для обработки нажатий на кнопки, а также задаются цвета кнопок.

Реализация конструктора приведена в листинге 10.

Листинг 10. Реализация конструктора InputWidget

```
1 InputWidget::InputWidget(QWidget* parent) : QDialog(parent) {
2
3     QLabel* surnameLabel = new QLabel("Surname:");
4     QLabel* nameLabel = new QLabel("Name:");
5     QLabel* middleNameLabel = new QLabel("Middle Name:");
6     QLabel* phoneLabel = new QLabel("Phones:");
7     QLabel* addressLabel = new QLabel("Address:");
8     QLabel* emailLabel = new QLabel("Email:");
9     QLabel* birthDateLabel = new QLabel("Birth Date:");
10
11     surnameEdit = new QLineEdit;
12     nameEdit = new QLineEdit;
13     middleNameEdit = new QLineEdit;
14     addressEdit = new QLineEdit;
15     emailEdit = new QLineEdit;
16     birthDateEdit = new QDateEdit;
17     birthDateEdit->setDisplayFormat("yyyy.MM.dd");
18
19     QPushButton* okButton = new QPushButton("OK");
20     connect(okButton, &QPushButton::clicked, this, [=]() {
21         if (validateData()) accept();
22     });
23     okButton->setStyleSheet(R"(
24     QPushButton {
25         background-color: #649B64;
26         color: white;
27     }
28     QPushButton:hover {
29         background-color: #3399ff;
30         color: black;
31     }
32     QPushButton:pressed {
33         background-color: #004080;
34     }
35     )");
```

```

36
37     QPushButton* addPhoneButton = new QPushButton("Add Phone");
38     connect(addPhoneButton, &QPushButton::clicked, this,
39     &InputWidget::addPhoneField);
40     addPhoneButton->setStyleSheet(R"(
41     QPushButton: hover {
42         background-color: #649B64;
43         color: black;
44     }
45     QPushButton: pressed {
46         background-color: #004080;
47     }
48     )" );
49
50
51     QFormLayout* formLayout = new QFormLayout;
52     formLayout->addRow(surnameLabel, surnameEdit);
53     formLayout->addRow(nameLabel, nameEdit);
54     formLayout->addRow(middleNameLabel, middleNameEdit);
55     formLayout->addRow(addressLabel, addressEdit);
56     formLayout->addRow(emailLabel, emailEdit);
57     formLayout->addRow(birthDateLabel, birthDateEdit);
58
59     phoneLayout = new QVBoxLayout;
60     formLayout->addRow(phoneLabel, phoneLayout);
61     phoneLayout->addWidget(addPhoneButton);
62     addPhoneField();
63
64     QHBoxLayout* buttonLayout = new QHBoxLayout;
65     buttonLayout->addWidget(okButton);
66     formLayout->addRow(buttonLayout);
67
68     setLayout(formLayout);
69 }

```

Деструктор SettingsWidget является деструктором по умолчанию, так как Qt сам управляет памятью, выделенной для виджетов.

Метод validateData проверяет введенные пользователем данные на соответствие заданным правилам с помощью регулярных выражений. В методе реализованы следующие ограничения:

- Фамилия и имя должны начинаться с заглавной буквы и содержать только буквы, пробелы, дефисы. Отчество может содержать пустое значение. Используемое регулярное выражение: `^[A-ZА-ЯЁ][\p{L}\s-]*[\p{L}]$` где `^` - начало строки; `[A-ZА-ЯЁ]` - одна заглавная (латинская или кириллическая) буква; `[\p{L}\s-]*` - любое количество букв, пробелов или дефисов; `[\p{L}]` - ограничение: строка должна заканчиваться буквой; `$` - конец строки. Для пустого значения отчества регулярное выражение `^-1$`, которое позволяет в начале строки находиться символу дефис ровно один раз.
- Адрес не может начинаться или заканчиваться знаками препинания. Используемое регулярное выражение: `^[\\p{L}].*[\\p{L}] $`, где `^` - начало

строки; `[\\pL]` - первая буква строки; `*` - любое количество любых символов; `[\\pL]` - последняя буква строки; `$` - конец строки.

- Email должно соответствовать стандартному формату адреса электронной почты. Используемое регулярное выражение:

`^([A-Za-z0-9]+([_ -][A-Za-z0-9]+)*)@([A-Za-z0-9-]+(\.[A-Za-z]{2,})+)$`,

где `^` - начало строки; `[A-Za-z0-9]+` - одна или несколько латинских букв или цифр; `([_ -][A-Za-z0-9]+)*` - ноль или более подстрок, начинающихся с символов `.`, `_`, `-`, за которыми следуют буквы или цифры; `@` - обязательный символ; `[A-Za-z0-9-]+` - одна или несколько латинских букв, цифр или дефисов; `([A-Za-z]{2,})+` - одна или более подстрок, начинающихся с точки, за которыми следуют две и более латинских букв; `$` - конец строки.

- Телефон проверяется с помощью метода `addPhoneField`.

Сообщение о каждой возможной ошибке при введении данных логируется с помощью функции класса `Logger`.

Реализация метода `validateData` приведена в листинге 11.

Листинг 11. Реализация метода `validateData`

```
1  bool InputWidget::validateData() {
2      QRegularExpression nameRegex("^([A-ZA-YE][\\p{L}\\s-]*[\\p{L}])$",
3      QRegularExpression::UseUnicodePropertiesOption);
4
5      QString surname = surnameEdit->text().trimmed();
6      if (!nameRegex.match(surname).hasMatch()) {
7          Logger::log("WARNING", "Invalid surname input: " + surname);
8          QMessageBox::warning(this, "Input error",
9          "Surname should start on capital letter and consist only letters,
10         numbers, hyphens and spaces");
11         return false;
12     }
13     surnameEdit->setText(surname);
14
15     QString name = nameEdit->text().trimmed();
16     if (!nameRegex.match(name).hasMatch()) {
17         Logger::log("WARNING", "Invalid name input: " + name);
18         QMessageBox::warning(this, "Input error",
19         "Name should start on capital letter and consist only letters,
20         numbers, hyphens and spaces");
21         return false;
22     }
23     nameEdit->setText(name);
24
25     QRegularExpression dashRegex("^-{1}$");
26     QString secname = middleNameEdit->text().trimmed();
27     if (secname.isEmpty()) {
28         secname = "-";
29     }
30     else if (!nameRegex.match(secname).hasMatch() && !dashRegex.match(
31     secname).hasMatch()) {
32         Logger::log("WARNING", "Invalid second name input: " + secname);
```

```

29     QMessageBox::warning(this, "Input error",
30     "Second name should start on capital letter and consist only
letters, numbers, hyphens and spaces");
31     return false;
32 }
33 middleNameEdit->setText(secname);
34
35 for (QLineEdit* phoneEdit : phoneEdits) {
36     if (phoneEdit->text().isEmpty() || !phoneEdit->hasAcceptableInput
37 ()) {
38         Logger::log("WARNING", "Invalid phone input: " + phoneEdit->text
39 ());
40         QMessageBox::warning(this, "Input error",
41         "Each phone should be in international standard. Example: +7 812
1234567 or 8(800)123-1212.");
42         return false;
43     }
44 }
45
46 QRegularExpression addressRegex("^([\\p{L}].[\\p{L}]$");
47 QString address = addressEdit->text().trimmed();
48 if (!addressRegex.match(address).hasMatch()) {
49     Logger::log("WARNING", "Invalid address input: " + address);
50     QMessageBox::warning(this, "Input error",
51     "Address should't start or and on punctuation marks");
52     return false;
53 }
54 addressEdit->setText(address);
55
56 QRegularExpression emailRegex(R"^(?([A-Za-z0-9]+([._-][A-Za-z0-9]+)*)
57 @([A-Za-z0-9-]+(\\.[A-Za-z]{2,})+)$)");
58 QString email = emailEdit->text().trimmed();
59 email.remove(QRegularExpression("\\s+"));
60 if (!emailRegex.match(email).hasMatch()) {
61     Logger::log("WARNING", "Invalid email input: " + email);
62     QMessageBox::warning(this, "Input error",
63     "Email should be a valid address (e.g., example@domain.com).");
64     return false;
65 }
66 emailEdit->setText(email);
67
68 return true;
69 }

```

Метод addPhoneField реализует добавление нового поля для ввода номера телефона. Он создает новое поле QLineEdit, которое с помощью регулярного выражения валидирует номер телефона, добавляет это поле в список phoneEdits для последующего сохранения. Для удаления лишних нечисловых символов используется сигнал editingFinished и вспомогательное регулярное выражение, удаляющее нечисловые символы.

Используемое регулярное выражение:

$\text{^\\+?\\d\\{1,4\\}?\\[\\s()\\-]*\\d\\{3\\}?\\[\\s()\\-]*\\d\\{3\\}?\\[\\s()\\-]*\\d\\{2,4\\}\\[\\s()\\-]*\\d+\\$}$,
где ^ - начало строки; $\text{\\+?}$ - необязательный символ +; $\text{\\d1,4?}$ - от 1 до 4 цифр (префикс выхода на международную линию); $\text{\\[\\s()\\-]*}$ - ноль или более пробелов, круглых скобок или дефисов; $\text{\\d3?}$ - три цифры (код страны); $\text{\\[\\s()\\-]*}$ - ноль или более пробелов, круглых скобок или дефисов;

`\\d3?` - три цифры (код города); `[\\s()-]*` - ноль или более пробелов, круглых скобок или дефисов; `\\d2, 4` - от двух до четырех цифр; `[\\s()-]*` - ноль или более пробелов, круглых скобок или дефисов; `\\d++` - одна или более цифр; `$` - конец строки.

Реализация метода `addPhoneNumber` приведена в листинге 12.

Листинг 12. Реализация метода `addPhoneField`

```
1 void InputWidget::addPhoneField() {
2     QLineEdit* phoneEdit = new QLineEdit(this);
3     phoneEdits.append(phoneEdit);
4     QRegularExpression phoneRegex("^\\+?\\d{1,4}?[\\s()-]*\\d{3}?[\\s()
5     phoneEdit->setValidator(new QRegularExpressionValidator(phoneRegex
6     connect(phoneEdit, &QLineEdit::editingFinished, this, [phoneEdit
7     {
8         QString phone = phoneEdit->text().remove(QRegularExpression("
9         phoneEdit->setText(phone);
10    });
11    phoneLayout->insertWidget(phoneLayout->count() - 1, phoneEdit);
12 }
```

Метод `setData` устанавливает данные в соответствующие поля таблицы в зависимости от индекса. Если телефонов больше, чем текущих полей, добавляются новые поля с помощью метода `addPhoneField`.

Реализация метода `setData` приведена в листинге 13.

Листинг 13. Реализация метода `setData`

```
1 void InputWidget::setData(int index, const QString& value) {
2     switch (index) {
3         case 0: surnameEdit->setText(value); break;
4         case 1: nameEdit->setText(value); break;
5         case 2: middleNameEdit->setText(value); break;
6         case 3: birthDateEdit->setDate(QDate::fromString(value, "yyyy.MM.dd"))
7         ; break;
8         case 4: addressEdit->setText(value); break;
9         case 5: {
10            QStringList phoneList = value.split(";");
11            for (const QString& phone : phoneList) {
12                if (phoneEdits.size() <= phoneList.indexOf(phone)) {
13                    addPhoneField();
14                }
15                phoneEdits[phoneList.indexOf(phone)]->setText(phone.trimmed());
16            }
17            break;
18        }
19        case 6: emailEdit->setText(value); break;
20    }
21 }
```

Метод `getData` позволяет получить данные из указанного поля по индексу, который передается на вход. Для нескольких телефонов применяется объединение через `;`.

Реализация метода `getData` приведена в листинге 14.

Листинг 14. Реализация метода `getData`

```
1 QString InputWidget::getData(int index) {
2     switch (index) {
3         case 0: return surnameEdit->text();
4         case 1: return nameEdit->text();
5         case 2: return middleNameEdit->text();
6         case 3: {
7             QString birthDate = birthDateEdit->date().toString("yyyy.MM.dd");
8             return birthDate;
9         }
10        case 4: return addressEdit->text();
11        case 5: {
12            QStringList phones;
13            for (QLineEdit* phoneEdit : phoneEdits) {
14                phones << phoneEdit->text();
15            }
16            return phones.join("; ");
17            return QString();
18        }
19        case 6: return emailEdit->text();
20    }
21 }
22
```

2.3 Класс `Logger`

Класс `Logger` предназначен для ведения логов в текстовом файле с различными уровнями логирования. В данном проекте используются два уровня логирования: `INFO` и `WARNING`. `INFO` предназначен для записи информационных сообщений, которые указывают на нормальное выполнение программы. Они полезны для отслеживания хода программы. С уровнем `INFO` добавляются сообщения о редактировании, удалении и добавлении записи. `WARNING` используется для записи предупреждающих сообщений, которые указывают на потенциальные проблемы, не являющиеся критическими. С уровнем `WARNING` добавляются сообщения о некорректном вводе данных, попытке удалить запись при пустом справочнике.

Метод `log` используется для записи сообщений логирования в текстовый файл. Он добавляет каждое сообщение с его временной меткой (дата и точное время) и уровнем логирования в специальный текстовый файл. Если файл не существует, он создается автоматически. Этот метод вызывается без необходимости создания экземпляра класса.

Реализация метода `log` приведена в листинге 15.

Листинг 15. Реализация метода `log`

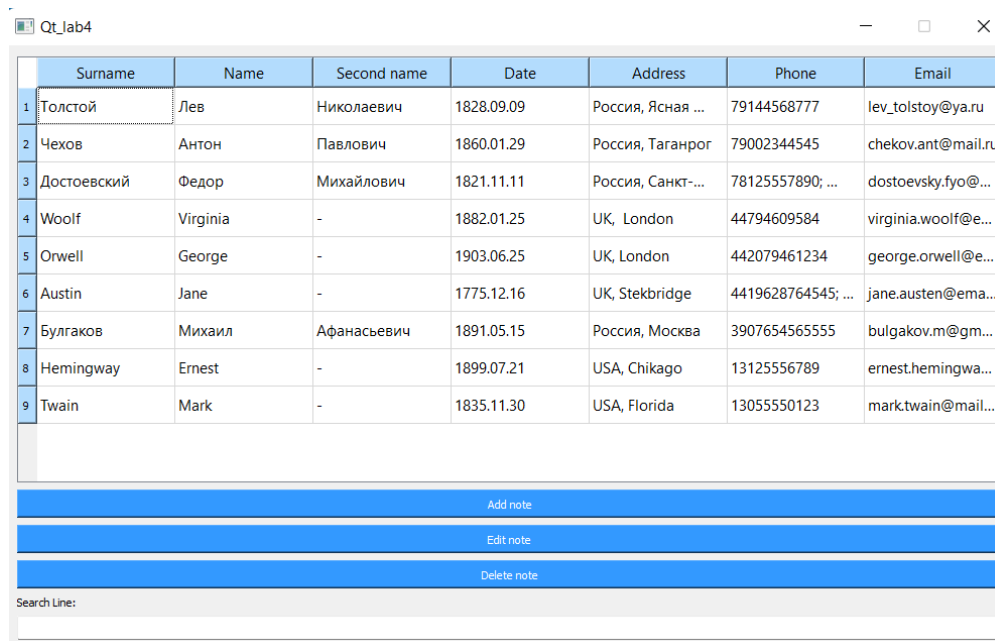
```
1 static void Logger::log(const QString& level, const QString& message)
2 {
3     QFile logFile("logs.txt");
4     if (logFile.open(QIODevice::Append | QIODevice::Text)) {
5         QTextStream out(&logFile);
6     }
7 }
```



```
5         out << QDateTime::currentDateTime().toString("yyyy-MM-dd HH:mm:ss.
    zzz")
6         << " [" << level << "]" " << message << "\n";
7         logFile.close();
8     }
9 }
10 };
11
```

3 Тестирование приложения

На рисунке 1 изображен стартовый экран приложения, на котором расположены все необходимые кнопки для управления. Данные, представленные в таблице, загружаются при запуске программы. На рисунке 2 представлено представление хранения данных в текстовом файле.

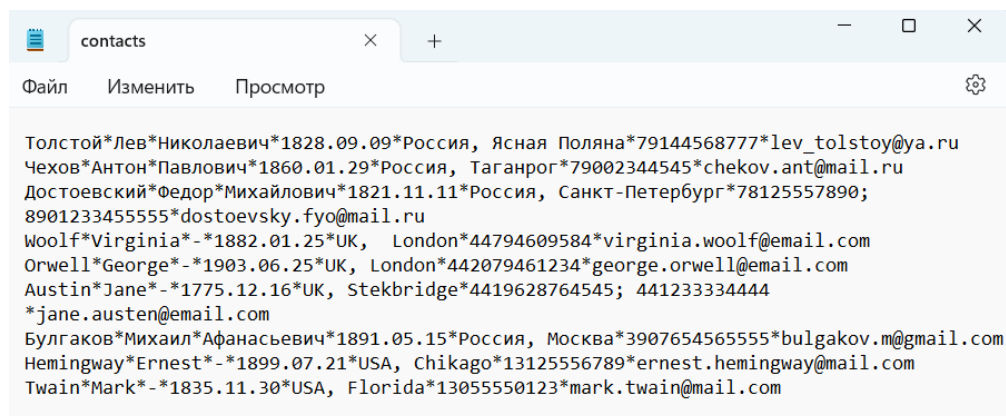


	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail.ru
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	UK, London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	UK, London	442079461234	george.orwell@e...
6	Austin	Jane	-	1775.12.16	UK, Stekbridge	4419628764545; ...	jane.austen@ema...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Россия, Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	USA, Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	USA, Florida	13055550123	mark.twain@mail...

Buttons: Add note, Edit note, Delete note

Search Line:

Рис. 1. Начальное окно



contacts

Файл Изменить Просмотр

Толстой*Лев*Николаевич*1828.09.09*Россия, Ясная Поляна*79144568777*lev_tolstoy@ya.ru
Чехов*Антон*Павлович*1860.01.29*Россия, Таганрог*79002344545*chekov.ant@mail.ru
Достоевский*Федор*Михайлович*1821.11.11*Россия, Санкт-Петербург*78125557890;
890123345555*dostoevsky.fyo@mail.ru
Woolf*Virginia*-*1882.01.25*UK, London*44794609584*virginia.woolf@email.com
Orwell*George*-*1903.06.25*UK, London*442079461234*george.orwell@email.com
Austin*Jane*-*1775.12.16*UK, Stekbridge*4419628764545; 441233334444
*jane.austen@email.com
Булгаков*Михаил*Афанасьевич*1891.05.15*Россия, Москва*390765456555*bulgakov.m@gmail.com
Hemingway*Ernest*-*1899.07.21*USA, Chikago*13125556789*ernest.hemingway@mail.com
Twain*Mark*-*1835.11.30*USA, Florida*13055550123*mark.twain@mail.com

Рис. 2. Хранение данных в документе

На рисунке 3 представлено диалоговое окно для ввода информации при попытке добавления нового контакта. На рисунке 4 отражен сценарий, когда пользователь неверно ввел какое-либо из значений и нажал подтверждающую кнопку. На рисунке 5 показано главное окно после добавления корректного контакта.

На рисунке 6 изображен процесс редактирования записи, на рисунке 7 представлена таблица с отредактированным значением.

Рис. 3. Диалоговое окно

Surname	Name	Birth Date	Address	Phone
Павлович		1860.01.29	Россия, Таганрог	79002344545
Михайлов			ия, Санкт-...	78125557890; .
ginia	-		London	44794609584
George				079461234
ne				628764545
ишаил				654565555
nest				5556789
ark				55550123

Рис. 4. Некорректно введенные данные

На рисунке 8 представлен результат сортировки по столбцу имени (по возрастанию).

На рисунке 9 отражен результат поиска по всем столбцам выражения, введенного в специальную строку. Все строки, которые не содержат этого выражения, скрыты.

На рисунке 10 представлено представление логированных записей в текстовом файле.

Qt_Jab4

	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail...
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	UK, London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	UK, London	442079461234	george.orwell@e...
6	Austin	Jane	-	1775.12.16	UK, Stekbridge	4419628764545; ...	jane.austen@em...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Россия, Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	USA, Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	USA, Florida	13055550123	mark.twain@mail...
10	Федоров	Федор	-	2000.01.01	г Санкт-Петербур	89112343434; ...	fedor@mail.com

Add note

Edit note

Delete note

Search Line:

Рис. 5. Вид главного окна после добавления

Qt_Jab4

	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail.ru
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	London	442079461234	george.orwell@e...
6	Austin	Jane	-	1775.12.16	Stekbridge	4419628764545; ...	jane.austen@ema...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	Florida	13055550123	mark.twain@mail...

Edit note

Delete note

Search Line:

Qt_Jab4 ? X

Surname: Austin

Name: Janett

Middle Name: -

Address: UK, Stekbridge

Email: jane.austen@email.com

Birth Date: 1775.12.16

Phones: 4419628764545

441233334444

Add Phone

OK

Рис. 6. Редактирование данных

Qt_Jab4

	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail.ru
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	UK, London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	UK, London	442079461234	george.orwell@e...
6	Austin	Janett	-	1775.12.16	UK, Stekbridge	4419628764545; ...	jane.austen@ema...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Россия, Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	USA, Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	USA, Florida	13055550123	mark.twain@mail...

Add note

Edit note

Delete note

Search Line:

Рис. 7. Обновленная таблица

Qt_Jab4

	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail.ru
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	London	442079461234	george.orwell@e...
6	Austin	Jane	-	1775.12.16	Stekbridge	4419628764545; ...	jane.austen@ema...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	Florida	13055550123	mark.twain@mail...

Edit note

Delete note

Search Line:

Qt_Jab4 ? X

Surname: Austin

Name: Janett

Middle Name: -

Address: UK, Stekbridge

Email: jane.austen@email.com

Birth Date: 1775.12.16

Phones: 4419628764545

441233334444

Add Phone

OK

Рис. 8. Сортировка данных по имени

Qt_lab4

	Surname	Name	Second name	Date	Address	Phone	Email
1	Толстой	Лев	Николаевич	1828.09.09	Россия, Ясная ...	79144568777	lev_tolstoy@ya.ru
2	Чехов	Антон	Павлович	1860.01.29	Россия, Таганрог	79002344545	chekov.ant@mail.ru
3	Достоевский	Федор	Михайлович	1821.11.11	Россия, Санкт-...	78125557890; ...	dostoevsky.fyo@...
4	Woolf	Virginia	-	1882.01.25	UK, London	44794609584	virginia.woolf@e...
5	Orwell	George	-	1903.06.25	UK, London	442079461234	george.orwell@e...
6	Austin	Janett	-	1775.12.16	UK, Stekbridge	4419628764545; ...	jane.austen@ema...
7	Булгаков	Михаил	Афанасьевич	1891.05.15	Россия, Москва	3907654565555	bulgakov.m@gm...
8	Hemingway	Ernest	-	1899.07.21	USA, Chikago	13125556789	ernest.hemingwa...
9	Twain	Mark	-	1835.11.30	USA, Florida	13055550123	mark.twain@mail...

Add note

Edit note

Delete note

Search Line:

Рис. 9. Результаты поиска

logs

Файл Изменить Просмотр

```

2024-11-20 13:20:24.602 [WARNING] Invalid name input: George23232
2024-11-20 13:20:45.366 [INFO] A note was successfully edited.
2024-11-20 13:21:00.744 [WARNING] Invalid surname input: hello
2024-11-20 13:21:34.690 [INFO] A new note was added to the table.
2024-11-20 13:21:42.046 [INFO] A row was successfully deleted from the table.

```

Рис. 10. Пример файла с логами

Заключение

В процессе выполнения практического задания была успешно реализована логика работы телефонного справочника. Все поставленные задачи были выполнены:

- 1) создана таблица для хранения данных и взаимодействия с полями, хранящими ФИО, адрес, дату рождения, email, телефонный адрес;
- 2) реализована проверка всех вводимых данных на корректность с помощью регулярных выражений. Проверка производится при каждом изменении значения поля;
- 3) реализована возможность чтения из файла формата .txt при запуске приложения и запись данных в файл при закрытии с помощью файла QFile;
- 4) реализована функция добавления новых записей с помощью диалогового окна с вводимыми данными;
- 5) реализована функция удаления выделенной строки с помощью нажатия кнопки deleteButton;
- 6) реализована функция редактирования выделенной строки с валидацией данных;
- 7) реализована сортировка данных по любому из существующих полей с помощью predefined сортировки класса QTableWidgetItem;
- 8) реализован поиск данных среди всех полей с помощью ввода поискового запроса и выявление тех строк, в которых было найдено частичное совпадение. по завершении поиска ячейки с информацией, не соответствующей поисковому запросу, скрываются;
- 9) реализовано логирование данных с помощью записи сообщений логирования в текстовый файл. Также записывается дата, точное время, уровень логирования.

Для разработки приложения была использована программа Visual Studio Code 2019 с компилятором MSVC 2017 для x64. Основной инструмент - фреймворк Qt версии 5.12.10.

Полный код приложения находится в приложении А.

Список использованной литературы

[1] Qt Assistant 5.12.10

[2] Шлее М. «Qt 5.10», БХВ-Петербург, 2018

Приложение А.

Листинг 16. Полный код приложения

```
1 //main.cpp
2 int main(int argc, char *argv[])
3 {
4     QApplication a(argc, argv);
5     MainWindow w;
6     w.show();
7     return a.exec();
8 }
9
10 //MainWindow.h
11 #pragma once
12 #include <QMainWindow>
13 #include <QTableWidget>
14 #include <QPushButton>
15 #include <QLineEdit>
16 #include <QFile>
17 #include <QVBoxLayout>
18 #include <QHeaderView>
19 #include <QMessageBox>
20 #include <QTextStream>
21 #include <QLabel>
22
23 class MainWindow : public QMainWindow
24 {
25     Q_OBJECT
26
27     public:
28     MainWindow(QWidget *parent = nullptr);
29     ~MainWindow() { saveToFile(); };
30
31     private:
32     QTableWidget* tableWidget;
33     QLineEdit* searchEdit;
34     bool ascendingOrder = true;
35     enum Column {Surname, Name, SecondName, Date, Address, Phone, Email,
36     ColumnCount };
37
38     void addEntry();
39     void editEntry();
40     void deleteEntry();
41     void searchEntry();
42     void sortEntries(int col);
43     void loadFromFile();
44     void saveToFile();
```

```

44     };
45
46
47     //MainWindow.cpp
48 #include "MainWindow.h"
49 #include "InputWidget.h"
50
51
52 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent){
53     setFixedSize(1000, 600);
54     tableWidget = new QTableWidget;
55     tableWidget->setColumnCount(ColumnCount);
56     tableWidget->setHorizontalHeaderLabels({"Surname", "Name", "Second name",
57     "Date", "Address", "Phone", "Email"});
58     tableWidget->horizontalHeader()->setSectionResizeMode(QHeaderView::
59     Stretch);
60
61     tableWidget->verticalHeader()->setStyleSheet("QHeaderView::section {
62     background-color: #B3DCFD; color: black; }");
63     tableWidget->horizontalHeader()->setStyleSheet("QHeaderView::section {
64     background-color: #B3DCFD; color: black; }");
65
66     QPalette palette = tableWidget->palette();
67     palette.setColor(QPalette::Highlight, QColor("#33FF99"));
68     palette.setColor(QPalette::HighlightedText, QColor("black"));
69     tableWidget->setPalette(palette);
70
71
72     QPushButton* addButton = new QPushButton("Add note");
73     addButton->setStyleSheet(R"(
74     QPushButton {
75         background-color: #3399ff;
76         color: white;
77     }
78     QPushButton:hover {
79         background-color: #02F7FD;
80         color: black;
81     }
82     QPushButton:pressed {
83         background-color: #004080;
84     }
85     )");
86     QPushButton* editButton = new QPushButton("Edit note");
87     editButton->setStyleSheet(R"(
88     QPushButton {
89         background-color: #3399ff;
90         color: white;
91     }
92     QPushButton:hover {
93         background-color: #24DB4E;

```

```

88     color: black;
89 }
90 QPushButton:pressed {
91     background-color: #004080;
92 }
93 ");
94 QPushButton* deleteButton = new QPushButton("Delete note");
95 deleteButton->setStyleSheet(R"(
96 QPushButton {
97     background-color: #3399ff;
98     color: white;
99 }
100 QPushButton:hover {
101     background-color: #EF9B10;
102     color: black;
103 }
104 QPushButton:pressed {
105     background-color: #DF9820;
106 }
107 )");
108 QLabel* searchLabel = new QLabel("Search Line:");
109 searchEdit = new QLineEdit;
110
111 QWidget* widget = new QWidget;
112 QVBoxLayout* layout = new QVBoxLayout(widget);
113 layout->addWidget(tableWidget);
114 layout->addWidget(addButton);
115 layout->addWidget(editButton);
116 layout->addWidget(deleteButton);
117 layout->addWidget(searchLabel);
118 layout->addWidget(searchEdit);
119
120 setCentralWidget(widget);
121
122 connect(tableWidget->horizontalHeader(), &QHeaderView::sectionClicked,
123         this, &MainWindow::sortEntries);
124 connect(addButton, &QPushButton::clicked, this, &MainWindow::addEntry);
125 connect(editButton, &QPushButton::clicked, this, &MainWindow::editEntry);
126 connect(deleteButton, &QPushButton::clicked, this, &MainWindow::deleteEntry);
127 connect(searchEdit, &QLineEdit::textChanged, this, &MainWindow::searchEntry);
128
129 loadFromFile();
130 }
131 void MainWindow::addEntry() {

```

```

132 InputWidget inputWidget(this);
133 if (inputWidget.exec() == QDialog::Accepted) {
134     tableWidget->insertRow(tableWidget->rowCount());
135     for (int i = Surname; i < ColumnCount; ++i) {
136         QTableWidgetItem* item = new QTableWidgetItem(inputWidget.getData(i)
137     );
138         item->setFlags(item->flags() & ~Qt::ItemIsEditable);
139         tableWidget->setItem(tableWidget->rowCount() - 1, i, item);
140     }
141     Logger::log("INFO", "A new note was added to the table.");
142 }
143
144 void MainWindow::editEntry() {
145     int row = tableWidget->currentRow();
146     if (row == -1) return;
147     InputWidget inputWidget(this);
148     for (int i = Surname; i < ColumnCount; ++i) {
149         inputWidget.setData(i, tableWidget->item(row, i)->text());
150     }
151     if (inputWidget.exec() == QDialog::Accepted) {
152         if (inputWidget.validateData()) {
153             for (int i = Surname; i < ColumnCount; ++i) {
154                 tableWidget->item(row, i)->setText(inputWidget.getData(i));
155             }
156             Logger::log("INFO", "A note was successfully edited.");
157         }
158     }
159 }
160
161 void MainWindow::deleteEntry() {
162     int row = tableWidget->currentRow();
163     if (row != -1)
164     {
165         tableWidget->removeRow(row);
166         Logger::log("INFO", "A row was successfully deleted from the table.");
167     }
168     else {
169         Logger::log("WARNING", "Attempt to delete a row when no row is
170         selected or no row exists.");
171     }
172 }
173
174 void MainWindow::searchEntry() {
175     QString query = searchEdit->text();
176     for (int i = 0; i < tableWidget->rowCount(); ++i) {
177         bool match = false;
178         for (int j = 0; j < tableWidget->columnCount(); ++j) {

```

```

178     if (tableWidget->item(i, j)->text().contains(query, Qt::
CaseInsensitive)) {
179         match = true;
180         break;
181     }
182 }
183 tableWidget->setRowHidden(i, !match);
184 }
185 }
186
187 void MainWindow::sortEntries(int column) {
188     tableWidget->sortItems(column, ascendingOrder ? Qt::AscendingOrder : Qt
::DescendingOrder);
189     ascendingOrder = !ascendingOrder;
190 }
191
192 void MainWindow::loadFromFile() {
193     QFile file("contacts.txt");
194     if (file.open(QIODevice::ReadOnly)) {
195         QTextStream in(&file);
196         in.setCodec("UTF-8");
197         while (!in.atEnd()) {
198             QStringList data = in.readLine().split("*");
199             if (data.size() == ColumnCount) {
200                 int row = tableWidget->rowCount();
201                 tableWidget->insertRow(row);
202                 for (int i = Surname; i < ColumnCount; ++i) {
203                     QTableWidgetItem* item = new QTableWidgetItem(data[i]);
204                     item->setFlags(item->flags() & ~Qt::ItemIsEditable);
205                     tableWidget->setItem(row, i, item);
206                 }
207             }
208         }
209         file.close();
210     }
211 }
212
213 void MainWindow::saveToFile() {
214     QFile file("contacts.txt");
215     if (file.open(QIODevice::WriteOnly)) {
216         QTextStream out(&file);
217         out.setCodec("UTF-8");
218         bool isRowComplete = true;
219         for (int i = 0; i < tableWidget->rowCount(); ++i) {
220             QStringList rowData;
221             for (int j = 0; j < tableWidget->columnCount(); ++j) {
222                 QTableWidgetItem* item = tableWidget->item(i, j);
223                 if (item && !item->text().isEmpty()) {

```

```

224         rowData << item->text();
225     }
226     else {
227         isRowComplete = false;
228         break;
229     }
230 }
231 if (isRowComplete) {
232     out << rowData.join("*") << "\n";
233 }
234 }
235 file.close();
236 }
237 }
238
239
240
241
242
243 //InputWidget.h
244 #pragma once
245
246 #include "MainWindow.h"
247 #include <QWidget>
248 #include <QDialog>
249 #include <QLineEdit>
250 #include <QDateEdit>
251 #include <QRegularExpression>
252 #include <QMessageBox>
253 #include <QFormLayout>
254
255 class InputWidget : public QDialog
256 {
257     Q_OBJECT
258 public:
259     explicit InputWidget(QWidget* parent = nullptr);
260     ~InputWidget() {};
261
262     QString getData(int index);
263     void setData(int index, const QString& value);
264     bool validateData();
265 private:
266     QLineEdit* surnameEdit, * nameEdit, * middleNameEdit, * emailEdit, *
addressEdit;
267     QDateEdit* birthDateEdit;
268     QList<QLineEdit*> phoneEdits;
269     QVBoxLayout* phoneLayout;
270

```

```

271     void addPhoneField();
272 };
273
274
275 //InputWidget.cpp
276 #include "InputWidget.h"
277
278 InputWidget::InputWidget(QWidget* parent) : QDialog(parent) {
279
280     QLabel* surnameLabel = new QLabel("Surname:");
281     QLabel* nameLabel = new QLabel("Name:");
282     QLabel* middleNameLabel = new QLabel("Middle Name:");
283     QLabel* phoneLabel = new QLabel("Phones:");
284     QLabel* addressLabel = new QLabel("Address:");
285     QLabel* emailLabel = new QLabel("Email:");
286     QLabel* birthDateLabel = new QLabel("Birth Date:");
287
288     surnameEdit = new QLineEdit;
289     nameEdit = new QLineEdit;
290     middleNameEdit = new QLineEdit;
291     addressEdit = new QLineEdit;
292     emailEdit = new QLineEdit;
293     birthDateEdit = new QDateEdit;
294     birthDateEdit->setDisplayFormat("yyyy.MM.dd");
295
296     QPushButton* okButton = new QPushButton("OK");
297     connect(okButton, &QPushButton::clicked, this, [=]() {
298         if (validateData()) accept();
299     });
300     okButton->setStyleSheet(R"(
301     QPushButton {
302         background-color: #649B64;
303         color: white;
304     }
305     QPushButton:hover {
306         background-color: #3399ff;
307         color: black;
308     }
309     QPushButton:pressed {
310         background-color: #004080;
311     }
312 )");
313
314     QPushButton* addPhoneButton = new QPushButton("Add Phone");
315     connect(addPhoneButton, &QPushButton::clicked, this,
316         &InputWidget::addPhoneField);
317     addPhoneButton->setStyleSheet(R"(
318     QPushButton:hover {

```

```

319     background-color: #649B64;
320     color: black;
321 }
322 QPushButton:pressed {
323     background-color: #004080;
324 }
325 )");
326
327
328 QFormLayout* formLayout = new QFormLayout;
329 formLayout->addRow(surnameLabel, surnameEdit);
330 formLayout->addRow(nameLabel, nameEdit);
331 formLayout->addRow(middleNameLabel, middleNameEdit);
332 formLayout->addRow(addressLabel, addressEdit);
333 formLayout->addRow(emailLabel, emailEdit);
334 formLayout->addRow(birthDateLabel, birthDateEdit);
335
336 phoneLayout = new QVBoxLayout;
337 formLayout->addRow(phoneLabel, phoneLayout);
338 phoneLayout->addWidget(addPhoneButton);
339 addPhoneField();
340
341 QHBoxLayout* buttonLayout = new QHBoxLayout;
342 buttonLayout->addWidget(okButton);
343 formLayout->addRow(buttonLayout);
344
345 setLayout(formLayout);
346 }
347
348
349 bool InputWidget::validateData() {
350     QRegularExpression nameRegex("[A-Z-A-YE][\\p{L}\\s-]*[\\p{L}]$",
351     QRegularExpression::UseUnicodePropertiesOption);
352
353     QString surname = surnameEdit->text().trimmed();
354     if (!nameRegex.match(surname).hasMatch()) {
355         Logger::log("WARNING", "Invalid surname input: " + surname);
356         QMessageBox::warning(this, "Input error",
357         "Surname should start on capital letter and consist only letters,
358 numbers, hyphens and spaces");
359         return false;
360     }
361     surnameEdit->setText(surname);
362
363     QString name = nameEdit->text().trimmed();
364     if (!nameRegex.match(name).hasMatch()) {
365         Logger::log("WARNING", "Invalid name input: " + name);
366         QMessageBox::warning(this, "Input error",

```



```

365     "Name should start on capital letter and consist only letters,
numbers, hyphens and spaces");
366     return false;
367 }
368 nameEdit->setText(name);
369
370 QRegularExpression dashRegex("^-{1}$");
371 QString secname = middleNameEdit->text().trimmed();
372 if (secname.isEmpty()) {
373     secname = "-";
374 }
375 else if (!nameRegex.match(secname).hasMatch() && !dashRegex.match(
secname).hasMatch()) {
376     Logger::log("WARNING", "Invalid second name input: " + secname);
377     QMessageBox::warning(this, "Input error",
378         "Second name should start on capital letter and consist only letters
, numbers, hyphens and spaces");
379     return false;
380 }
381 middleNameEdit->setText(secname);
382
383 for (QLineEdit* phoneEdit : phoneEdits) {
384     if (phoneEdit->text().isEmpty() || !phoneEdit->hasAcceptableInput())
{
385         Logger::log("WARNING", "Invalid phone input: " + phoneEdit->text()
);
386         QMessageBox::warning(this, "Input error",
387             "Each phone should be in international standard. Example: +7 812
1234567 or 8(800)123-1212.");
388         return false;
389     }
390 }
391
392 QRegularExpression addressRegex("^([\\p{L}].*[\\p{L}])$");
393 QString address = addressEdit->text().trimmed();
394 if (!addressRegex.match(address).hasMatch()) {
395     Logger::log("WARNING", "Invalid address input: " + address);
396     QMessageBox::warning(this, "Input error",
397         "Address should't start or end on punctuation marks");
398     return false;
399 }
400 addressEdit->setText(address);
401
402 QRegularExpression emailRegex(R"^(?([A-Za-z0-9]+([._-][A-Za-z0-9]+)*)@
([A-Za-z0-9-]+(\\.[A-Za-z]{2,})+)$)");
403 QString email = emailEdit->text().trimmed();
404 email.remove(QRegularExpression("\\s+"));
405 if (!emailRegex.match(email).hasMatch()) {

```

```

406     Logger::log("WARNING", "Invalid email input: " + email);
407     QMessageBox::warning(this, "Input error",
408         "Email should be a valid address (e.g., example@domain.com).");
409     return false;
410 }
411 emailEdit->setText(email);
412
413     return true;
414 }
415
416 void InputWidget::addPhoneField() {
417     QLineEdit* phoneEdit = new QLineEdit(this);
418     phoneEdits.append(phoneEdit);
419     QRegularExpression phoneRegex("^\\+?\\d{1,4}?[\\s()-]*\\d{3}?[\\s()-]*\\d{3}?[\\s()-]*\\d{2,4}[\\s()-]*\\d+$");
420     phoneEdit->setValidator(new QRegularExpressionValidator(phoneRegex,
421         this));
422     connect(phoneEdit, &QLineEdit::editingFinished, this, [phoneEdit]() {
423         QString phone = phoneEdit->text().remove(QRegularExpression("[^\\d]"));
424         phoneEdit->setText(phone);
425     });
426     phoneLayout->insertWidget(phoneLayout->count() - 1, phoneEdit);
427 }
428
429 void InputWidget::setData(int index, const QString& value) {
430     switch (index) {
431         case 0: surnameEdit->setText(value); break;
432         case 1: nameEdit->setText(value); break;
433         case 2: middleNameEdit->setText(value); break;
434         case 3: birthDateEdit->setDate(QDate::fromString(value, "yyyy.MM.dd")); break;
435         case 4: addressEdit->setText(value); break;
436         case 5: {
437             QStringList phoneList = value.split(";");
438             for (const QString& phone : phoneList) {
439                 if (phoneEdits.size() <= phoneList.indexOf(phone)) {
440                     addPhoneField();
441                 }
442                 phoneEdits[phoneList.indexOf(phone)]->setText(phone.trimmed());
443             }
444             break;
445         }
446         case 6: emailEdit->setText(value); break;
447     }
448 }
449
450 QString InputWidget::getData(int index) {

```

```

450     switch (index) {
451         case 0: return surnameEdit->text();
452         case 1: return nameEdit->text();
453         case 2: return middleNameEdit->text();
454         case 3: {
455             QString birthDate = birthDateEdit->date().toString("yyyy.MM.dd");
456             return birthDate;
457         }
458         case 4: return addressEdit->text();
459         case 5: {
460             QStringList phones;
461             for (QLineEdit* phoneEdit : phoneEdits) {
462                 phones << phoneEdit->text();
463             }
464             return phones.join("; ");
465             return QString();
466         }
467         case 6: return emailEdit->text();
468     }
469 }
470
471 \\Logger.h
472 #pragma once
473 #include <QFile>
474 #include <QTextStream>
475 #include <QDateTime>
476
477
478 class Logger
479 {
480 public:
481     static void log(const QString& level, const QString& message) {
482         QFile logFile("logs.txt");
483         if (logFile.open(QIODevice::Append | QIODevice::Text)) {
484             QTextStream out(&logFile);
485             out << QDateTime::currentDateTime().toString("yyyy-MM-dd HH:mm:ss.
486 zzz")
487             << " [" << level << "]" << message << "\n";
488             logFile.close();
489         }
490     }
491 };
492
493

```